

UAS
PENGOLAHAN CITRA



NAMA : Muhamad Afdhal Mukti

NIM : 202431010

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom.

NO.PC : 4

ASISTEN : 1. JOSHUA INDRA PRATAMA

2. DAVINA NAJWA ERMAWAN

3. IZZAT ISLAMI KAGAPI

4. SAKURA AMASTASYA SALSABILA SETIYANTO

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

Contents

DAFTAR ISI 2

BAB I 3

PENDAHULUAN..... 3

1.1 Rumusan Masalah 3

1.2 Tujuan Masalah..... 3

1.3 Manfaat Masalah..... 3

BAB II 4

LANDASAN TEORI 4

BAB III 7

HASIL..... 7

BAB IV 11

PENUTUP 11

DAFTAR PUSTAKA 12

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana mengimplementasikan metode Canny Edge Detection untuk mendeteksi tepi pada citra digital?
2. Bagaimana menerapkan Contour Detection untuk memperoleh bentuk atau kontur objek dari hasil deteksi tepi?
3. Bagaimana menghasilkan keluaran berupa citra asli, citra hasil deteksi tepi, dan citra hasil contour detection menggunakan OpenCV?

1.2 Tujuan Masalah

1. Mengimplementasikan metode Canny Edge Detection untuk mendeteksi tepi objek pada citra digital.
2. Mengimplementasikan Contour Detection untuk memperoleh kontur dari objek yang telah dideteksi.
3. Menampilkan hasil pengolahan citra berupa gambar asli (Original Image), hasil Canny Edge Detection, dan hasil Contour Detection.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai konsep dasar pengolahan citra digital, khususnya deteksi tepi dan deteksi kontur.
2. Meningkatkan kemampuan dalam mengimplementasikan algoritma pengolahan citra menggunakan Python dan OpenCV.
3. Memberikan pengalaman dalam melakukan analisis bentuk objek berdasarkan hasil deteksi tepi.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Citra digital pada dasarnya adalah kumpulan piksel yang disusun dalam matriks, di mana tiap piksel menyimpan nilai intensitas warna. Pengolahan citra digital adalah proses mengubah atau menganalisis citra tersebut lewat komputer supaya informasinya lebih jelas, misalnya untuk memperbaiki kualitas gambar atau memisahkan objek dari latar belakangnya. Tahapannya biasanya berurutan: mulai dari akuisisi gambar, pra-pemrosesan seperti konversi warna dan penghalusan noise, lalu segmentasi, ekstraksi fitur, sampai akhirnya citra bisa diinterpretasikan atau diklasifikasikan.

Pada project ini, tahapan yang benar-benar dipakai cuma sebagian, yaitu konversi ke grayscale, penghalusan pakai Gaussian blur, deteksi tepi Canny, dan diakhiri dengan pencarian kontur objek. Semua proses ini dikerjakan pakai OpenCV di Python, sebuah pustaka computer vision open source yang memang banyak dipakai untuk riset maupun aplikasi pengolahan citra karena koleksi fungsinya lengkap, mulai dari operasi dasar sampai deteksi

2.2 Ruang Warna (RGB, Grayscale, HSV)

Foto yang diambil kamera pada dasarnya tersimpan dalam ruang warna RGB, yaitu kombinasi tiga kanal warna merah, hijau, dan biru. Sebelum masuk ke deteksi tepi, citra biasanya dikonversi dulu ke grayscale karena algoritma seperti Canny bekerja berdasarkan perubahan intensitas di satu kanal saja, jadi lebih ringan dihitung dan hasilnya konsisten. Selain grayscale, ada juga ruang warna HSV yang kadang dipakai kalau grayscale-nya kurang kontras, karena komponen saturation atau value bisa menonjolkan bagian yang warnanya mirip dengan latar belakang, misalnya kulit dengan warna tembok

2.3 Gaussian Blur

Gaussian blur dipakai untuk meredam noise sebelum citra masuk ke tahap deteksi tepi. Caranya dengan mengonvolusi citra memakai kernel berbentuk distribusi Gaussian, di mana piksel yang posisinya dekat dengan pusat kernel diberi bobot lebih besar. Hasilnya, variasi intensitas kecil akibat noise bisa diredam tanpa merusak bentuk tepi utama secara drastis.

Ukuran kernel yang dipakai sangat menentukan hasil akhirnya. Kernel kecil kurang mampu meredam tekstur baju atau dinding, sementara kernel yang kebesaran malah bisa menghapus tepi tipis yang justru dibutuhkan, misalnya tepi lengan yang kontrasnya sudah rendah dari sisanya. Gaussian filter cukup efektif dipakai sebagai tahap awal karena mengurangi noise acak tanpa banyak mengubah bentuk objek. Sebagai alternatif, ada juga median filter yang lebih cocok untuk noise salt-and-pepper, dan bilateral filter yang bisa meredam noise

sambil tetap menjaga ketajaman tepi. Pemilihan salah satu dari ketiganya perlu disesuaikan dengan jenis noise dan seberapa penting detail tepi yang mau dipertahankan.

2.4 Jenis-Jenis Metode Deteksi Tepi

Dalam pengolahan citra digital terdapat beberapa algoritma yang digunakan untuk mendeteksi tepi. Setiap algoritma memiliki karakteristik yang berbeda sesuai dengan kebutuhan pengolahan citra.

Metode Sobel menggunakan operator gradien untuk mendeteksi perubahan intensitas pada arah horizontal dan vertikal. Metode ini cukup sederhana dan memiliki proses komputasi yang cepat, namun masih cukup sensitif terhadap noise.

Metode Prewitt memiliki konsep yang hampir sama dengan Sobel, yaitu menghitung perubahan intensitas berdasarkan gradien citra. Perbedaannya terletak pada nilai kernel yang digunakan sehingga hasil deteksi dapat sedikit berbeda.

Metode Laplacian menggunakan turunan orde kedua untuk menemukan perubahan intensitas pada citra. Metode ini mampu menghasilkan garis tepi yang lebih detail, tetapi sangat peka terhadap gangguan noise sehingga biasanya didahului dengan proses peredaman noise.

Metode Canny merupakan salah satu algoritma deteksi tepi yang paling banyak digunakan karena mampu menghasilkan garis tepi yang tipis, jelas, dan memiliki tingkat kesalahan yang relatif rendah dibandingkan metode lainnya.

2.5 Algoritma Canny Edge Detection

Algoritma Canny Edge Detection dikembangkan oleh John F. Canny pada tahun 1986 dan hingga saat ini masih menjadi salah satu metode standar dalam pengolahan citra digital. Algoritma ini dirancang untuk menghasilkan deteksi tepi yang optimal dengan mempertimbangkan tiga kriteria utama, yaitu mampu mendeteksi sebanyak mungkin tepi yang sebenarnya, meminimalkan kesalahan deteksi, dan menghasilkan posisi tepi yang tepat.

Proses kerja algoritma Canny terdiri atas beberapa tahapan. Tahap pertama adalah mengurangi noise menggunakan Gaussian Blur. Proses ini dilakukan agar gangguan kecil pada citra tidak ikut terdeteksi sebagai garis tepi.

Tahap berikutnya adalah menghitung gradien intensitas untuk mengetahui perubahan nilai piksel pada setiap bagian citra. Dari hasil gradien tersebut akan diperoleh arah dan besar perubahan intensitas.

Selanjutnya dilakukan proses Non-Maximum Suppression untuk mempertahankan hanya piksel yang memiliki nilai gradien terbesar sehingga garis tepi yang dihasilkan menjadi lebih tipis.

Tahap berikutnya adalah Double Threshold yang berfungsi untuk membedakan tepi kuat dan tepi lemah berdasarkan nilai ambang tertentu.

Tahap terakhir adalah Edge Tracking by Hysteresis, yaitu menghubungkan tepi lemah yang masih memiliki hubungan dengan tepi kuat sehingga menghasilkan garis tepi yang lebih utuh dan tidak terputus.

Melalui tahapan tersebut, algoritma Canny mampu menghasilkan kualitas deteksi tepi yang lebih baik dibandingkan sebagian besar metode konvensional.

2.6 Faktor yang Mempengaruhi Hasil Deteksi Tepi

Kualitas hasil deteksi tepi dipengaruhi oleh beberapa faktor. Salah satunya adalah kualitas citra yang digunakan. Citra dengan resolusi tinggi dan pencahayaan yang baik akan menghasilkan deteksi tepi yang lebih akurat.

Selain itu, keberadaan noise juga mempengaruhi hasil deteksi. Noise dapat menyebabkan munculnya garis-garis yang sebenarnya bukan merupakan bagian dari objek sehingga diperlukan proses penghalusan citra sebelum dilakukan deteksi tepi.

Nilai threshold yang digunakan pada algoritma Canny juga sangat menentukan hasil akhir. Nilai threshold yang terlalu rendah dapat menyebabkan banyak noise terdeteksi sebagai tepi, sedangkan threshold yang terlalu tinggi dapat menghilangkan sebagian garis tepi objek.

Posisi objek, kontras antara objek dan latar belakang, serta kondisi pencahayaan saat pengambilan gambar juga menjadi faktor penting yang mempengaruhi keberhasilan proses deteksi tepi.

2.7 Penerapan Deteksi Tepi

Deteksi tepi memiliki berbagai kegunaan dalam kehidupan sehari-hari maupun di bidang industri. Pada bidang medis, deteksi tepi digunakan untuk membantu mengidentifikasi batas organ pada hasil pencitraan seperti CT Scan dan MRI. Pada bidang keamanan, teknik ini dimanfaatkan dalam sistem pengenalan wajah serta pendeteksian objek pada kamera pengawas.

Di bidang industri, deteksi tepi digunakan untuk proses inspeksi kualitas produk secara otomatis, misalnya mendeteksi bentuk atau ukuran suatu komponen. Selain itu, pada sistem kendaraan otonom, deteksi tepi dimanfaatkan untuk mengenali marka jalan, rambu lalu lintas, maupun objek di sekitar kendaraan.

BAB III

HASIL

```
: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

- import cv2 as cv, import numpy as np, dan import matplotlib.pyplot as plt merupakan library yang digunakan dalam program pengolahan citra digital. Library OpenCV (cv2) berfungsi sebagai alat utama untuk membaca, mengolah, dan menampilkan citra, seperti melakukan deteksi tepi, segmentasi, maupun berbagai proses pengolahan gambar lainnya. Library NumPy (numpy) digunakan untuk mengolah data dalam bentuk array atau matriks, karena citra digital pada dasarnya tersusun dari kumpulan nilai piksel. Sementara itu, Matplotlib (pyplot) dimanfaatkan untuk menampilkan hasil pengolahan citra dalam bentuk gambar maupun grafik, seperti histogram, sehingga hasil yang diperoleh dapat diamati dan dianalisis

```
: image = cv.imread("foto.jpeg")
image_rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
```

- Kode `image = cv.imread("foto.jpeg")` digunakan untuk membaca file gambar bernama `foto.jpeg` dan menyimpannya ke dalam variabel `image`. Secara bawaan, OpenCV membaca gambar dengan urutan warna BGR (Blue, Green, Red). Oleh karena itu, pada baris berikutnya digunakan fungsi `cv.cvtColor(image, cv.COLOR_BGR2RGB)` untuk mengubah format warna dari BGR menjadi RGB (Red, Green, Blue). Perubahan ini diperlukan agar gambar dapat ditampilkan dengan warna yang sesuai ketika menggunakan Matplotlib, karena library tersebut mengenali format warna RGB sebagai standar penampilannya.

```
# Ubah foto menjadi hitam putih
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
blurred = cv.GaussianBlur(gray, (15, 15), 0)

edges = cv.Canny(blurred, 38, 80)

contours, hierarchy = cv.findContours(edges, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
contour_image = image_rgb.copy()

cv.drawContours(contour_image, contours, -1, (0, 255, 0), 2)
```

- Kode di atas digunakan untuk melakukan proses deteksi kontur pada sebuah gambar. Langkah pertama adalah mengubah gambar berwarna menjadi citra grayscale menggunakan fungsi `cv.cvtColor()`, sehingga proses pengolahan menjadi lebih sederhana karena hanya menggunakan satu kanal warna. Selanjutnya, citra dihaluskan dengan Gaussian Blur menggunakan kernel berukuran (15, 15) untuk mengurangi noise yang dapat mengganggu proses deteksi tepi. Setelah itu, fungsi `cv.Canny()` digunakan untuk mendeteksi tepi objek dengan nilai ambang bawah 38 dan ambang atas 80. Hasil deteksi tepi kemudian diproses menggunakan `cv.findContours()` untuk mencari kontur terluar pada objek yang terdapat di dalam gambar. Sebelum kontur digambar, salinan dari gambar asli dalam format RGB dibuat agar gambar awal tidak

berubah. Terakhir, fungsi `cv.drawContours()` digunakan untuk menggambar seluruh kontur yang berhasil dideteksi dengan warna hijau dan ketebalan garis 2 piksel, sehingga bentuk objek dapat terlihat dengan jelas.

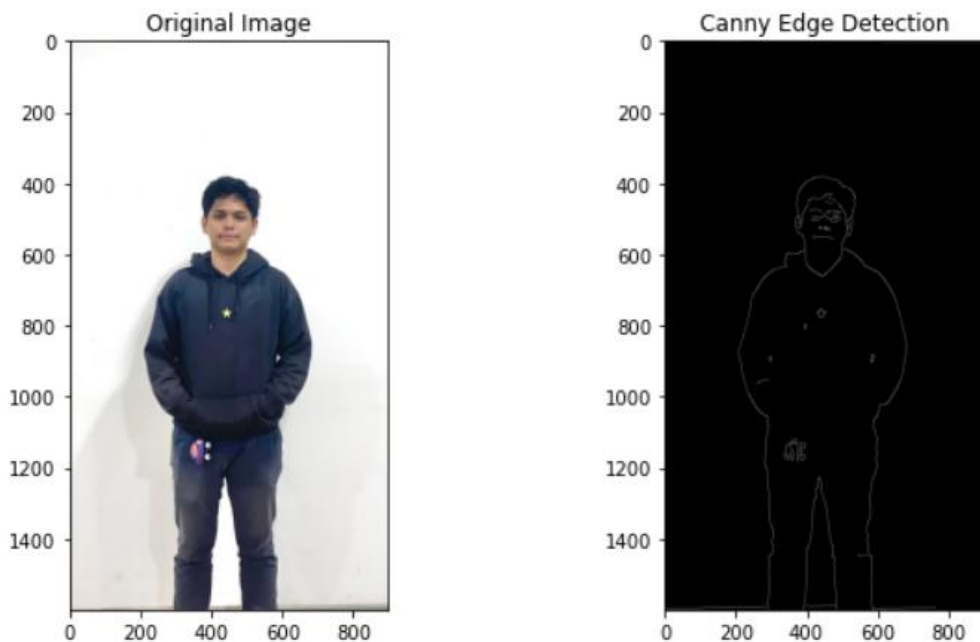
```
print("Output 1")
fig1, axs1 = plt.subplots(1, 2, figsize=(10, 5))

axs1[0].imshow(image_rgb)
axs1[0].set_title('Original Image')

axs1[1].imshow(edges, cmap="gray")
axs1[1].set_title('Canny Edge Detection')

plt.tight_layout()
plt.show() # Tampilkan gambar bagian atas
```

Output 1



- Output 1 menampilkan dua gambar yang disusun bersebelahan untuk memudahkan perbandingan. Gambar di sebelah kiri merupakan citra asli (Original Image) yang telah dikonversi ke format RGB sehingga warna tampil dengan benar saat ditampilkan menggunakan Matplotlib. Sementara itu, gambar di sebelah kanan merupakan hasil Canny Edge Detection, yaitu proses deteksi tepi yang menampilkan batas-batas objek dalam bentuk garis berwarna putih pada latar belakang hitam. Dari hasil tersebut dapat terlihat bahwa hanya bagian tepi objek yang dipertahankan, sedangkan area lain dihilangkan. Tampilan ini menjadi dasar untuk proses pencarian kontur pada tahap berikutnya.

```
print("Output 2")
fig2, axs2 = plt.subplots(1, 3, figsize=(15, 5))

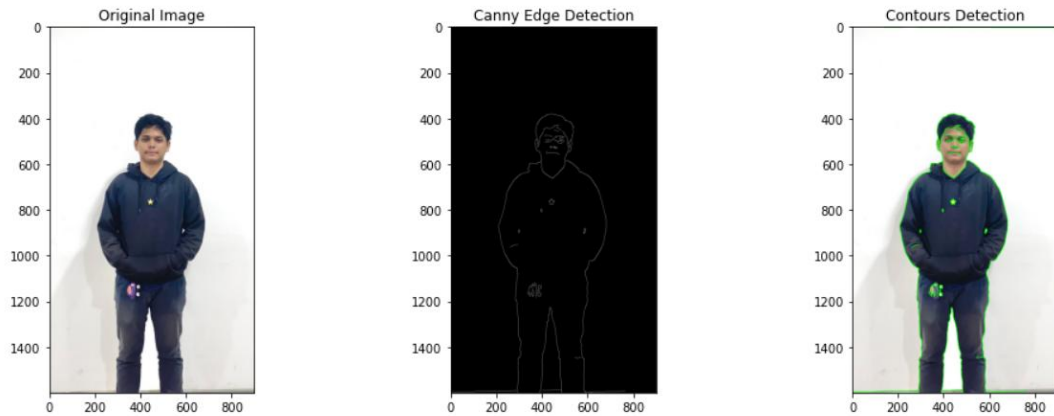
axs2[0].imshow(image_rgb)
axs2[0].set_title('Original Image')

axs2[1].imshow(edges, cmap="gray")
axs2[1].set_title('Canny Edge Detection')

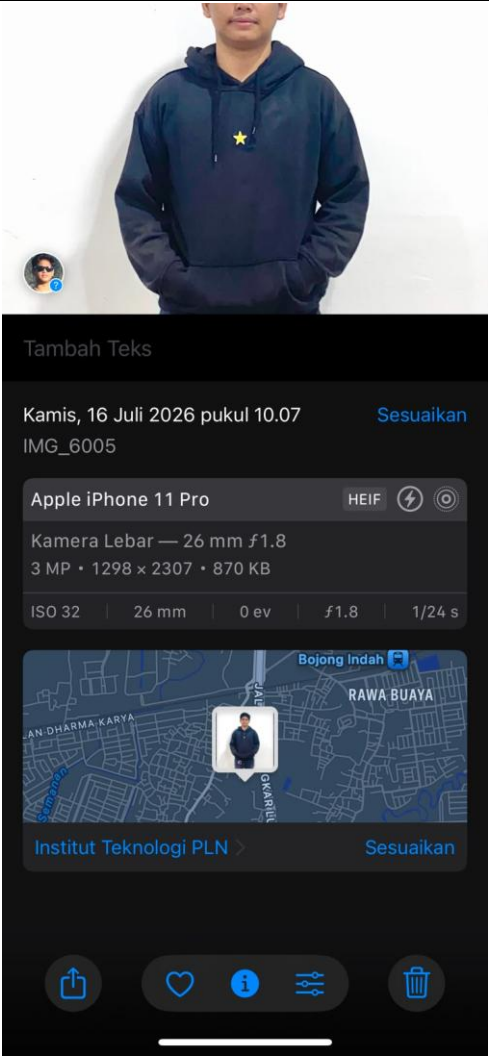
axs2[2].imshow(contour_image)
axs2[2].set_title('Contours Detection')

plt.tight_layout()
plt.show()
```

Output 2



- Output 2 menampilkan tiga gambar secara berdampingan, yaitu citra asli, hasil deteksi tepi menggunakan metode Canny, dan hasil deteksi kontur. Gambar pertama merupakan citra asli yang digunakan sebagai acuan. Gambar kedua memperlihatkan hasil deteksi tepi, di mana hanya batas-batas objek yang terlihat dalam bentuk garis putih pada latar belakang hitam. Selanjutnya, gambar ketiga menampilkan hasil Contours Detection, yaitu garis kontur berwarna hijau yang digambar mengikuti batas luar objek berdasarkan hasil deteksi tepi sebelumnya. Dengan membandingkan ketiga gambar tersebut, dapat dilihat bahwa proses deteksi kontur berhasil mengenali bentuk objek dan menampilkannya pada citra asli sehingga objek menjadi lebih mudah diamati dan dianalisis.



BAB IV

PENUTUP

Berdasarkan UAS praktikum yang telah dilaksanakan, dapat disimpulkan bahwa proses deteksi garis tepi pada citra digital berhasil dilakukan menggunakan metode Canny Edge Detection. Metode ini mampu menampilkan batas-batas objek pada gambar dengan cukup jelas sehingga bentuk objek dapat dikenali dengan lebih mudah. Setelah proses deteksi tepi dilakukan, hasilnya kemudian digunakan untuk memperoleh kontur objek sehingga bentuk luar objek dapat divisualisasikan dengan lebih baik.

Selama praktikum juga diperoleh pemahaman mengenai tahapan pengolahan citra, mulai dari pembacaan citra, proses pengurangan noise, deteksi garis tepi, hingga pendeteksian kontur objek. Hasil yang diperoleh menunjukkan bahwa kualitas citra, pencahayaan, serta pemilihan parameter pada algoritma Canny berpengaruh terhadap ketepatan hasil deteksi. Semakin baik kualitas citra yang digunakan, maka hasil deteksi garis tepi dan kontur akan semakin akurat.

Melalui UAS praktikum ini, tujuan pembelajaran mengenai penerapan metode deteksi garis tepi pada pengolahan citra digital telah tercapai. Selain menambah pemahaman terhadap konsep edge detection.

DAFTAR PUSTAKA

1. Saluky, & Marine, Y. (2023). Penerapan algoritma deteksi tepi Canny menggunakan Python dan OpenCV. *Smart Techno (Smart Technology Informatics and Technopreneurship)*, 5(1), 1–7.
2. Prayogi, D., Lubis, S. R., Rizko, M. A., & Suteja, A. G. (2025). Optimalisasi segmentasi citra digital metode Canny edge detection dan thresholding. *RIGGS: Journal of Artificial Intelligence and Digital Business*.
3. Khoirani, L., Ariansyah, R., & Supiyandi. (2024). Aplikasi pengolahan citra untuk peningkatan deteksi tepi melalui segmentasi citra. *Mars: Jurnal Teknik Mesin, Industri, Elektro dan Ilmu Komputer*, 2(3).